ONTENT

S.NO	TOPIC	PAGE NO
1	INTRODUCTION	1
2	OBJECTIVE	2
3	PROBLEM DEFINATION	3
4	FEATURES	4
5	FLOW CHART	5
6	HARDWARE	6
7	SOFTWARE	7
8	DATABASE DESIGN	8
9	PYTHON-SQL CONNCETIVITY	9-10
10	SOURCE CODE	11-14
11	OUTPUT	15-17
12	CONCLUSION	18
13	BIBLIOGRAPHY	19

INTRODUCTION

A Sales Management System is a computer-based application developed to manage and maintain sales-related information efficiently. In many shops and small businesses, sales records are often maintained manually, which can be time-consuming and may lead to errors in calculations and record keeping.

This project provides a simple and user-friendly solution for managing products, stock, customers, and sales transactions. The system is developed using **Python** as the programming language and **MySQL** as the database.

The application allows the user to add and view products, search for products, record sales, update stock automatically, view sales records, and calculate total sales. The use of a database ensures that information is stored systematically and can be accessed whenever required.

This project demonstrates the practical use of **Python programming, SQL queries, database management, and Python-MySQL connectivity** to solve a real-life problem.

OBJECTIVE

The main objective of the Sales Management System is to provide a simple and efficient computerized system for managing sales and product records.

The objectives of this project are:

1. To maintain product details such as product name, category, price, and stock.
2. To record sales transactions and customer information.
3. To automatically calculate the total amount of each sale.
4. To update product stock after every sale.
5. To make searching and viewing records easier.
6. To reduce errors and manual work in maintaining sales records.
7. To store information securely using a MySQL database.
8. To demonstrate the use of Python with SQL connectivity.
9. To develop a simple and user-friendly solution to a real-life business problem.

PROBLEM DEFINITION

Managing sales records manually can be difficult, especially when a business has a large number of products and customers. Maintaining product details, stock quantities, customer information, and sales transactions in notebooks or registers can result in errors, loss of records, and difficulty in finding old information.

Calculating the total amount of sales manually can also be time-consuming and may lead to calculation mistakes. Updating stock after every sale is another important task that requires accuracy.

Therefore, there is a need for a simple computerized system that can store and manage sales information efficiently. The **Sales Management System** provides a solution by using **Python and MySQL** to store product and sales records, calculate transaction amounts, update stock, and retrieve information quickly whenever required.

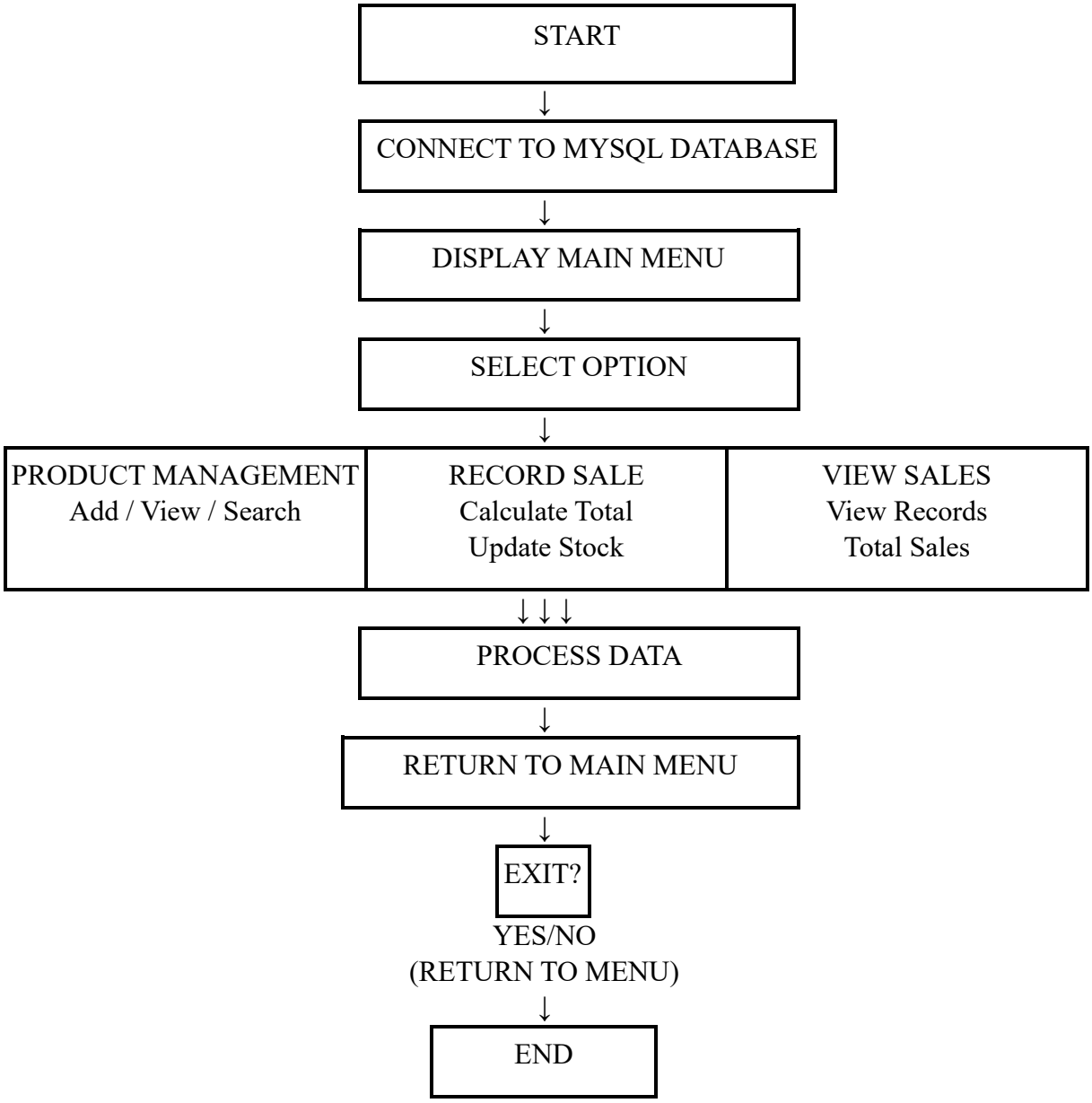
FEATURES

The main features of the Sales Management System are:

1. Product Management – Add, view, and search product details.
2. Stock Management – Maintain and automatically update the available stock.
3. Sales Recording – Record customer sales transactions with quantity and date.
4. Automatic Calculation – Calculate the total amount of each sale automatically.
5. Sales Records – View previously recorded sales transactions.
6. Product Search – Quickly search for products using their names.
7. Total Sales – Calculate and display the total sales amount.
8. Database Storage – Store product and sales information securely in MySQL.
9. User-Friendly Interface – Simple menu-based interface that is easy to operate.
10. Reduced Manual Work – Minimizes errors and saves time compared with manual record keeping.

FLOWCHART

SALES MANAGEMENT SYSTEM



HARDWARE SPECIFICATION

Minimum Hardware Requirements

- **Processor:** Intel Core i3 or equivalent
- **RAM:** 4 GB or above
- **Storage:** 500 MB free space
- **Keyboard:** Standard keyboard
- **Mouse:** Standard mouse
- **Monitor:** Standard display

SOFTWARE SPECIFICATION

- **Operating System:** Windows 11
- **Programming Language:** Python 3
- **Database:** MySQL
- **Database Connector:** mysql-connector-python
- **IDE:** IDLE

- **Terminal:** Windows Command Prompt

DATABASE DESIGN

Product Table

Field	Data Type	Description
product_id	INT	Primary key
product_name	VARCHAR(100)	Product name
category	VARCHAR(50)	Product category
price	DECIMAL(10,2)	Product price
stock	INT	Available quantity

Sales Table

Field	Data Type	Description
sale_id	INT	Primary key
product_ic	INT	Product ID
customer_na	VARCHAR(1(	Customer name
quantity	INT	Quantity sold
total_amou	DECIMAL(10	Total sale
sale_date	DATE	Date of sale

SQL COMMAND

DATA BASE DESIGN

TABLE PRODUCTS

```sql
USE sales_management;
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    price DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL
);
```

TABLE SALES

```sql
CREATE TABLE sales (
    sale_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    customer_name VARCHAR(100),
    quantity INT NOT NULL,
    total_amount DECIMAL(10,2),
    sale_date DATE,
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

PRODUCT

```sql

INSERT INTO products (product_name, category, price, stock)
VALUES
('Pen', 'Stationery', 10.00, 100),
('Notebook', 'Stationery', 50.00, 50),
('Water Bottle', 'Accessories', 150.00, 30),
('School Bag', 'Bags', 800.00, 20),
('Calculator', 'Electronics', 500.00, 15);
SELECT * FROM products;
DELETE FROM products
WHERE product_id > 5;
SELECT * FROM products;
USE sales_management;
```

PRODUCT SALES

```
42 •  INSERT INTO sales
43    (product_id, customer_name, quantity, total_amount, sale_date)
44    VALUES
45    (1, 'Rahul', 3, 30.00, '2026-08-21'),
46    (2, 'Priya', 2, 100.00, '2026-08-21'),
47    (3, 'Aman', 1, 150.00, '2026-08-21'),
48    (4, 'Riya', 1, 800.00, '2026-08-21'),
49    (5, 'Arjun', 2, 1000.00, '2026-08-21');
50 •  SELECT * FROM sales;
51 •  DELETE FROM sales
52    WHERE sale_id > 5;
53 •  SELECT * FROM sales;
```

SOURCE CODE

DATABASE CONNECTION

```
# ---------- DATABASE CONNECTION ----------
def connect_database():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="kamiruj",
        database="sales_management"
    )
```

ADD PRODUCT

```
# ---------- ADD PRODUCT ----------
def add_product():
    conn = connect_database()
    cursor = conn.cursor()

    name = input("Enter product name: ")
    category = input("Enter category: ")

    try:
        price = float(input("Enter price: "))
        stock = int(input("Enter stock quantity: "))

        query = """
        INSERT INTO products (product_name, category, price, stock)
        VALUES (%s, %s, %s, %s)
        """

        cursor.execute(query, (name, category, price, stock))
        conn.commit()

        print("\nProduct added successfully!")

    except ValueError:
        print("\nPlease enter valid numbers for price and stock.")

    finally:
        cursor.close()
        conn.close()
```

VIEW PRODUCT

```
# ---------- VIEW PRODUCTS ----------
def view_products():
    conn = connect_database()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM products")
    products = cursor.fetchall()

    print("\n========== PRODUCTS ==========")
    print("ID | Product | Category | Price | Stock")
    print("-----------------------------------------")

    for product in products:
        print(
            product[0], "|",
            product[1], "|",
            product[2], "| ₹",
            product[3], "|",
            product[4]
        )

    cursor.close()
    conn.close()
```

MAKE SALE

```
# ---------- MAKE SALE ----------
def make_sale():
    conn = connect_database()
    cursor = conn.cursor()

    try:
        product_id = int(input("Enter product ID: "))

        cursor.execute(
            "SELECT product_name, price, stock FROM products WHERE product_id = %s",
            (product_id,)
        )

        product = cursor.fetchone()

        if product is None:
            print("\nProduct not found!")
            return

        product_name, price, stock = product

        print("\nProduct:", product_name)
        print("Price: ₹", price)
        print("Available stock:", stock)

        customer = input("Enter customer name: ")
        quantity = int(input("Enter quantity: "))

        if quantity <= 0:
            print("\nQuantity must be greater than zero.")
            return

        if quantity > stock:
            print("\nNot enough stock available!")
            return

        total = price * quantity

        sale_query = """
        INSERT INTO sales
        (product_id, customer_name, quantity, total_amount, sale_date)
        VALUES (%s, %s, %s, %s, %s)
        """

        cursor.execute(
            sale_query,
            (product_id, customer, quantity, total, date.today())
        )

        update_query = """
        UPDATE products
        SET stock = stock - %s
        WHERE product_id = %s
        """

        cursor.execute(update_query, (quantity, product_id))

        conn.commit()

        print("\n========== SALE SUCCESSFUL ==========")
        print("Product:", product_name)
        print("Customer:", customer)
        print("Quantity:", quantity)
        print("Total Amount: ₹", total)
        print("Date:", date.today())

    except ValueError:
        print("\nPlease enter valid numbers.")

    finally:
        cursor.close()
        conn.close()
```

VIEW SALE

```
# ---------- VIEW SALES ----------
def view_sales():
    conn = connect_database()
    cursor = conn.cursor()

    query = """
    SELECT sales.sale_id,
           products.product_name,
           sales.customer_name,
           sales.quantity,
           sales.total_amount,
           sales.sale_date
    FROM sales
    JOIN products
    ON sales.product_id = products.product_id
    """

    cursor.execute(query)
    sales = cursor.fetchall()

    print("\n================ SALES RECORDS ================")
    print("ID | Product | Customer | Qty | Amount | Date")
    print("-----------------------------------------------")

    for sale in sales:
        print(
            sale[0], "|",
            sale[1], "|",
            sale[2], "|",
            sale[3], "| ₹",
            sale[4], "|",
            sale[5]
        )

    cursor.close()
    conn.close()
```

SEARCH PRODUCT

```
# ---------- SEARCH PRODUCT ----------
def search_product():
    conn = connect_database()
    cursor = conn.cursor()

    name = input("Enter product name to search: ")
```

OUTPUT

ADD PRODUCT

```
======================================
       SALES MANAGEMENT SYSTEM
======================================
1. Add Product
2. View Products
3. Make Sale
4. View Sales
5. Search Product
6. Exit
======================================
Enter your choice: 1
Enter product name: CLASSSMATE NOTEBOOK
Enter category: STATIONERY
Enter price: 80
Enter stock quantity: 25

Product added successfully!
```

VIEW PRODUCT

```
======================================
       SALES MANAGEMENT SYSTEM
======================================
1. Add Product
2. View Products
3. Make Sale
4. View Sales
5. Search Product
6. Exit
======================================
Enter your choice: 2

========== PRODUCTS ==========
ID | Product | Category | Price | Stock
-----------------------------------------
1 | Pen | Stationery | ₹ 10.00 | 100
2 | Notebook | Stationery | ₹ 50.00 | 50
3 | Water Bottle | Accessories | ₹ 150.00 | 28
4 | School Bag | Bags | ₹ 800.00 | 20
5 | Calculator | Electronics | ₹ 500.00 | 15
41 | pen | stationery | ₹ 10.00 | 10
42 | CLASSSMATE NOTEBOOK | STATIONERY | ₹ 80.00 | 25
```

MAKE SALE

```
======================================
       SALES MANAGEMENT SYSTEM
======================================
1. Add Product
2. View Products
3. Make Sale
4. View Sales
5. Search Product
6. Exit
======================================
Enter your choice: 3
Enter product ID: 42

Product: CLASSSMATE NOTEBOOK
Price: ₹ 80.00
Available stock: 25
Enter customer name: kamiruj
Enter quantity: 3

========== SALE SUCCESSFUL ==========
Product: CLASSSMATE NOTEBOOK
Customer: kamiruj
Quantity: 3
Total Amount: ₹ 240.00
Date: 2026-08-21
```

VIEW SALES

```
======================================
       SALES MANAGEMENT SYSTEM
======================================
1. Add Product
2. View Products
3. Make Sale
4. View Sales
5. Search Product
6. Exit
======================================
Enter your choice: 4

================ SALES RECORDS ================
ID | Product | Customer | Qty | Amount | Date
-----------------------------------------------
1 | Pen | Rahul | 3 | ₹ 30.00 | 2026-08-21
2 | Notebook | Priya | 2 | ₹ 100.00 | 2026-08-21
3 | Water Bottle | Aman | 1 | ₹ 150.00 | 2026-08-21
4 | School Bag | Riya | 1 | ₹ 800.00 | 2026-08-21
5 | Calculator | Arjun | 2 | ₹ 1000.00 | 2026-08-21
21 | Water Bottle | aman | 2 | ₹ 300.00 | 2026-08-21
22 | CLASSSMATE NOTEBOOK | kamiruj | 3 | ₹ 240.00 | 2026-08-21
```

SEARCH PRODUCT

```
======================================
       SALES MANAGEMENT SYSTEM
======================================
1. Add Product
2. View Products
3. Make Sale
4. View Sales
5. Search Product
6. Exit
======================================
Enter your choice: 5
Enter product name to search: Pen

========== SEARCH RESULTS ==========
ID: 1
Product: Pen
Category: Stationery
Price: ₹ 10.00
Stock: 100
----------------------------------
ID: 41
Product: pen
Category: stationery
Price: ₹ 10.00
Stock: 10
----------------------------------
```

EXIT

```
======================================
       SALES MANAGEMENT SYSTEM
======================================
1. Add Product
2. View Products
3. Make Sale
4. View Sales
5. Search Product
6. Exit
======================================
Enter your choice: 6

Thank you for using Sales Management System!
```

CONCLUSION

The Sales Management System is a simple and efficient computerized solution for managing product and sales records. It helps in storing product details, recording sales, updating stock, and calculating total sales accurately.

The project successfully demonstrates the use of **Python programming and MySQL database connectivity** to solve a real-life problem. It reduces manual work, saves time, minimizes calculation errors, and makes sales information easier to manage and retrieve.

Through this project, I gained practical knowledge of **Python, SQL, database management, and Python-MySQL connectivity**. The system can also be further improved by adding features such as user login, billing, report generation, and a graphical user interface.

BIBLIOGRAPHY

1. Python Documentation – Python programming language and functions.
2. MySQL Documentation – SQL commands, databases, and table management.
3. MySQL Connector/Python Documentation – Python and MySQL connectivity.
4. CBSE Computer Science Textbook – Concepts of Python and database management.